The Natural Operating System: Boolean Computation via Möbius Inversion and Prime-Encoded Databases

Alexander Pisani

a.h.pisani.research@gmail.com

March 2026

Abstract

We present a computational architecture in which Boolean logic emerges from the multiplicative structure of the integers via the Möbius function. Inputs are encoded as sets of primes; programs are stored at squarefree integer addresses; program activation is governed by divisibility; and the Möbius signs automatically implement inclusion-exclusion for Boolean logic. The architecture is governed by a single master equation:

$$y(S) = \sum_{n \in D} \mu(n) \cdot O_n \cdot [n \mid S] \cdot P_n(S)$$

where μ is the Möbius function, O_n is the database output, $[n \mid S]$ is a divisibility-based activation indicator, and $P_n(S)$ is a condition-testing processor.

We prove that the sum is always finite, verify that the four primitives (μ, activation, processor, database) are each computable using only GCD, logarithm, and arithmetic — without external conditional logic — and demonstrate that NOT, AND, OR, and NAND gates are all implementable as database programs. Since NAND is functionally complete, any Boolean function can be realised within the framework. We further show that the Euler product structure $R(S) = \prod_{p \in S}(1 - 1/p^2)$ is itself a program in the master equation with output values $O_n = 1/n^2$, unifying the "operating system" with the programs it runs.

As an optional application, we derive a factor-free computational identity: the factorisation fingerprint $\rho(N) = \zeta(2) \cdot \prod_{p|N}(1 - 1/p^2)$ equals the localised zeta function $\zeta_N(2) = \sum_{\gcd(n,N)=1} n^{-2}$, which is computable via GCD operations alone without knowledge of the factors of N.

Keywords: Möbius function, inclusion-exclusion, Boolean logic, prime encoding, Euler product, divisibility, functional completeness

MSC (2020): 11A25, 03B05, 68Q05, 11M06

---

1. Introduction

The Möbius function $\mu(n)$ and the Euler product $\zeta(s) = \prod_p (1 - p^{-s})^{-1}$ are classical objects in analytic number theory. The Möbius inversion formula, which uses μ to "undo" summation over

divisors, is a standard tool whose signs implement the inclusion-exclusion principle over prime sets. This paper takes a computational perspective on these classical objects: we show that they naturally provide the structural components of an architecture for Boolean computation.

The architecture — which we call the *Natural Operating System* — works as follows. An input is a finite set S of primes, encoded from a binary string. A *database* D assigns to each squarefree integer address n an output value O_n and a condition C_n. A *program* at address n is *activated* when all prime factors of n divide the input — that is, when n divides the integer $\prod_{p \in S} p$. The Möbius function $\mu(n)$ provides the sign (additive or subtractive) for each activated program. A processor function $P_n(S)$ tests whether a secondary condition is met.

The master equation that governs the entire computation is:

$$y(S) = \sum_{n \in D} \mu(n) \cdot O_n \cdot [n \mid S] \cdot P_n(S)$$

The main results of the paper are:

1. **Primitive verification** (Section 3): Each of the four components — $\mu(n)$, activation $[n \mid S]$, processor $P_n(S)$, and database O_n — is computable using only GCD, logarithm, prime factorisation, and arithmetic. No external conditional logic is required.
2. **Finiteness** (Section 4): The sum $y(S)$ has finitely many nonzero terms for any finite input and finite database.
3. **Functional completeness** (Section 5): The Boolean gates NOT, AND, OR, and NAND are all implementable as database programs. Since NAND is functionally complete, any Boolean function can be realised.
4. **Möbius as inclusion-exclusion** (Section 6): The Möbius signs $\mu(n)$ automatically provide the correct coefficients for the inclusion-exclusion principle. This is not a design choice but a structural consequence of the Euler product inverse.
5. **Native computation** (Section 7): The partial Euler product $R(S) = \prod_{p \in S}(1 - 1/p^2)$ is itself a program in the master equation, with $O_n = 1/n^2$ and trivial condition $C_n = 1$. The "operating system" and the "programs" share the same Möbius structure.

Relation to prior work. This paper builds on the algebraic framework of [1], which establishes the monoid isomorphism $\Omega(1/n) = \ln(n)$ and the four-layer correspondence among number theory, Boolean logic, resistance topology, and prime coordinate vectors. The present paper uses that framework as its mathematical foundation and extends it to an explicit computational architecture governed by the master equation.

What this paper does not claim. We do not claim that the architecture offers computational advantages over conventional Boolean circuit models. The framework provides a different *language* for describing Boolean computation — one grounded in the multiplicative structure of the integers — but whether this perspective yields practical benefits beyond conceptual insight is an open question (see Section 10).

1.1 Organisation

Section 2 establishes notation and recalls essential background. Section 3 rigorously defines and verifies the four primitives. Section 4 proves finiteness. Section 5 presents the Boolean gate imple-

mentations with complete verification tables. Section 6 explains the structural role of the Möbius function. Section 7 shows that native computation is itself a program. Section 8 demonstrates conditional logic and gate composition. Section 9 presents the factor-free fingerprint identity as an application. Section 10 discusses limitations and open problems.

---

2. Mathematical Preliminaries

2.1 The Resistance Isomorphism (Summary)

We recall from [1] that the map $\Omega(1/n) = \ln(n)$ is a monoid isomorphism from inverse integers under multiplication to non-negative reals under addition. This map is unique up to positive scale (Cauchy's functional equation). We use the notation $\Omega_n = \ln(n)$ throughout.

2.2 The Möbius Function

Definition 2.1. The *Möbius function* $\mu\colon \mathbb{Z}^+ \to \{-1, 0, 1\}$ is defined by:

$$\mu(n) = \begin{cases} 1 & \text{if } n = 1, \\ (-1)^k & \text{if } n = p_1 p_2 \cdots p_k \text{ is squarefree with } k \text{ distinct prime factors}, \\ 0 & \text{if } n \text{ has a squared prime factor.} \end{cases}$$

The Möbius function satisfies the inversion identity $\sum_{d|n} \mu(d) = [n = 1]$, which is the algebraic content of the inclusion-exclusion principle applied to prime divisors.

2.3 The Euler Product Inverse

The Dirichlet series of μ gives the inverse of the Riemann zeta function:

$$\frac{1}{\zeta(s)} = \sum_{n=1}^{\infty} \frac{\mu(n)}{n^s} = \prod_{p \text{ prime}} (1 - p^{-s}), \qquad \mathrm{Re}(s) > 1.$$

At $s = 2$, the product converges to $6/\pi^2$, the probability that two randomly chosen integers are coprime [2].

2.4 Input Encoding

Definition 2.2 (Binary-to-Prime Encoding). Let p_i denote the i-th prime (0-indexed: $p_0 = 2$, $p_1 = 3$, $p_2 = 5$, etc.). The encoding function $\beta\colon \{0,1\}^* \to \mathcal{P}(\text{Primes})$ maps a binary string $b_k b_{k-1} \cdots b_1 b_0$ to the prime set:

$$\beta(b_k \cdots b_0) = \{p_i : b_i = 1\}.$$

Examples: $\beta(\texttt{11}) = \{2, 3\}$; $\beta(\texttt{1011}) = \{2, 3, 7\}$; $\beta(\texttt{11011}) = \{2, 3, 7, 11\}$.

Remark. This encoding is a convention requiring a prime table. Once the set S is obtained, all subsequent computation is pure arithmetic. The encoding itself is not part of the mathematical content of the architecture.

---

3. The Four Primitives

The master equation $y(S) = \sum_{n \in D} \mu(n) \cdot O_n \cdot [n \mid S] \cdot P_n(S)$ contains four primitive components. In this section we rigorously define each, verify that it is computable without external conditional logic, and provide its resistance interpretation following [1].

3.1 Primitive 1: $\mu(n)$ — The Routing Function

The Möbius function $\mu(n)$ is computed from the prime factorisation of n:

> **Algorithm.** Factor n. If any prime appears with exponent > 1, return 0. Otherwise, count the number k of distinct prime factors and return $(-1)^k$.

$\mu(n)$	Meaning	Circuit interpretation
$+1$	Additive contribution	In-phase signal
-1	Subtractive contribution	π-shifted signal (phase interference)
0	Invalid address (not squarefree)	Blocked: infinite resistance

The phase connection is exact: $\mu(n) = e^{i\pi k}$ where k is the number of distinct prime factors of n.

Verification (selected values):

n	Factorisation	Squarefree?	k	$\mu(n)$
1	(empty)	Yes	0	$+1$
2	2	Yes	1	-1
3	3	Yes	1	-1
4	2^2	No	—	0
6	2×3	Yes	2	$+1$
30	$2 \times 3 \times 5$	Yes	3	-1

3.2 Primitive 2: $[n \mid S]$ — The Activation Indicator

Definition 3.1 (Activation for Set Input). For a squarefree integer n and a prime set S:

$$[n \mid S] = \begin{cases} 1 & \text{if every prime factor of } n \text{ is in } S, \\ 0 & \text{otherwise.} \end{cases}$$

For finite integer input $x = \prod_{p \in S} p^{a_p}$, this reduces to a divisibility test: $[n \mid x] = 1$ iff n divides x.

Conditional-free implementation. For integer inputs:

$$[n \mid x] = \left\lfloor \frac{\gcd(n, x)}{n} \right\rfloor.$$

Proof. We have $\gcd(n, x) = n$ if and only if $n \mid x$. When $n \mid x$, the ratio is 1 and $\lfloor 1 \rfloor = 1$. When $n \nmid x$, we have $\gcd(n, x) < n$, so $\gcd(n, x)/n < 1$ and $\lfloor \gcd(n, x)/n \rfloor = 0$. □

Resistance interpretation:

$$\Omega_{\text{act}}(n, x) = \ln\left(\frac{n}{\gcd(n, x)}\right).$$

When $n \mid x$: $\Omega_{\text{act}} = 0$ (zero resistance — signal passes). When $n \nmid x$: $\Omega_{\text{act}} > 0$ (positive resistance — signal blocked).

Verification:

n	x	$\gcd(n, x)$	$\gcd/n$	$[n \mid x]$
6	12	6	1.000	1
6	14	2	0.333	0
7	49	7	1.000	1
7	50	1	0.143	0

3.3 Primitive 3: $P_n(S)$ — The Processor Function

The processor tests whether a condition is met by the input. We define two versions:

Binary processor (primary). For a prime condition C:

$$P(C, S) = \begin{cases} 1 & \text{if } C \in S, \\ 0 & \text{if } C \notin S. \end{cases}$$

For trivial condition $C = 1$: $P(1, S) = 1$ always.

Continuous processor (generalisation). For integer condition $C > 1$ and integer input x:

$$P(C, x) = \frac{\ln(\gcd(C, x))}{\ln(C)}.$$

This yields: $P = 1$ when $C \mid x$ (condition fully met); $P = 0$ when $\gcd(C, x) = 1$ (coprime, no match); and $0 < P < 1$ for partial overlap (when $1 < \gcd(C, x) < C$).

Resistance interpretation:

$$\Omega_{\text{proc}}(C, x) = \ln\left(\frac{C}{\gcd(C, x)}\right).$$

This is zero when $C \mid x$ (condition met, zero resistance) and positive otherwise.

Remark on the two versions. For the Boolean computation developed in this paper, the binary processor is sufficient and yields clean $\{0, 1\}$ outputs. The continuous processor is a natural generalisation that arises from the resistance framework and produces fractional values for partial matches. It may be of interest for non-Boolean computation but is not needed for the results of this paper.

Verification (continuous version):

C	x	$\gcd(C, x)$	$P(C, x)$	Interpretation
7	14	7	1.000	Condition met
7	10	1	0.000	No match
11	22	11	1.000	Condition met
6	15	3	0.613	Partial (61.3%)

3.4 Primitive 4: O_n — The Database Output

Definition 3.2 (Database). A database D is a function mapping squarefree integer addresses to (output, condition) pairs:

$$D(n) = (O_n, C_n)$$

where $O_n \in \mathbb{R}$ is the output value and C_n is the condition (a prime, or 1 for a trivial condition). The convention $O_n = 0$ means "no program at address n." The *support* of the database is the finite set $\{n : O_n \neq 0\}$.

Circuit interpretation. Each program at address n is a series circuit with two gates:

```
Input x  [ Activation Gate ] [ Processor Gate ] > Output
             [n|x]?               [C_n|x]?

Signal flows when BOTH gates pass (series connection).
Final output weighted by  (n).
```

---

4. The Finiteness Theorem

Theorem 4.1. For any finite database D and any input S (equivalently, any finite integer x), the sum

$$y(S) = \sum_{n=1}^{\infty} \mu(n) \cdot O_n \cdot [n \mid S] \cdot P_n(S)$$

has finitely many nonzero terms.

Proof. The term at position n is nonzero only if all of the following hold:

1. $\mu(n) \neq 0$ (i.e., n is squarefree);
2. $O_n \neq 0$ (a program exists at address n);
3. $[n \mid S] = 1$ (all prime factors of n are in S);
4. $P_n(S) \neq 0$ (the condition has nonzero overlap with the input).

By condition 2, only finitely many n satisfy $O_n \neq 0$. By condition 3, only finitely many n divide x (since x has at most $d(x)$ divisors). The nonzero terms are indexed by the intersection $\{n : O_n \neq 0\} \cap \{n : n \mid x\}$, which is finite as the intersection of two finite sets. □

---

5. Boolean Gate Implementations

In this section we demonstrate that the master equation implements the standard Boolean gates. The input convention is: bit A is ON iff $2 \in S$ (equivalently, $2 \mid x$); bit B is ON iff $3 \in S$ (equivalently, $3 \mid x$).

5.1 NOT Gate

Goal: Output 1 when A is OFF; output 0 when A is ON.

Derivation. We seek $\text{NOT}(A) = 1 - [2 \mid x]$. Using $\mu(1) = +1$ and $\mu(2) = -1$:

- Address $n = 1$: term $= \mu(1) \cdot 1 \cdot [1 \mid x] \cdot 1 = +1$.
- Address $n = 2$: term $= \mu(2) \cdot 1 \cdot [2 \mid x] \cdot 1 = -[2 \mid x]$.

Total: $y = 1 - [2 \mid x] = \text{NOT}(A)$.

Database:

n	O_n	C_n	$\mu(n)$	Role
1	1	1	$+1$	Reference signal ($+1$)
2	1	1	-1	Subtract when $2 \mid x$

Verification:

x	A	$[1 \mid x]$	$[2 \mid x]$	$y(x)$	$\text{NOT}(A)$
1	0	1	0	1	1
2	1	1	1	0	0
3	0	1	0	1	1
6	1	1	1	0	0

Circuit interpretation. The NOT gate operates by *phase interference*: a constant reference signal ($+1$ from address 1) and a π-shifted input signal ($-[2 \mid x]$ from address 2, where $\mu(2) = -1$) combine to produce the complement. This is the same mechanism by which destructive interference produces cancellation in wave physics.

5.2 AND Gate

Goal: Output 1 when both A and B are ON.

Derivation. The key insight is that $[6 \mid x] = 1$ if and only if both $2 \mid x$ and $3 \mid x$, since $6 = 2 \times 3$. That is, the AND condition is *encoded in the composite address*.

Since $\mu(6) = +1$: the single term $\mu(6) \cdot 1 \cdot [6 \mid x] \cdot 1 = [6 \mid x]$.

Database:

n	O_n	C_n	$\mu(n)$	Role
6	1	1	$+1$	Output 1 when $6 \mid x$

Verification:

x	A	B	$[6 \mid x]$	$y(x)$	$\text{AND}(A, B)$
1	0	0	0	0	0
2	1	0	0	0	0
3	0	1	0	0	0
6	1	1	1	1	1

Circuit interpretation. The AND gate uses *series activation*: the composite address $6 = 2 \times 3$ naturally requires both prime factors to be present in x. This is the divisibility-lattice expression of conjunction.

5.3 OR Gate

Goal: Output 1 when A or B (or both) are ON.

Derivation. By the inclusion-exclusion principle:

$$\text{OR}(A, B) = A + B - A \cdot B = [2 \mid x] + [3 \mid x] - [6 \mid x].$$

We need three terms with coefficients $+1, +1, -1$. The Möbius values are $\mu(2) = -1$, $\mu(3) = -1$, $\mu(6) = +1$. Setting $O_n = -1$ for all three addresses:

- Address 2: $\mu(2) \cdot (-1) \cdot [2 \mid x] = (-1)(-1)[2 \mid x] = +[2 \mid x]$.
- Address 3: $\mu(3) \cdot (-1) \cdot [3 \mid x] = (-1)(-1)[3 \mid x] = +[3 \mid x]$.
- Address 6: $\mu(6) \cdot (-1) \cdot [6 \mid x] = (+1)(-1)[6 \mid x] = -[6 \mid x]$.

Total: $y = [2 \mid x] + [3 \mid x] - [6 \mid x] = \text{OR}(A, B)$.

Database:

n	O_n	C_n	$\mu(n)$	$\mu \cdot O$	Role
2	-1	1	-1	$+1$	Add $[2 \mid x]$
3	-1	1	-1	$+1$	Add $[3 \mid x]$
6	-1	1	$+1$	-1	Subtract $[6 \mid x]$ (overlap correction)

Verification:

x	A	B	$[2 \mid x]$	$[3 \mid x]$	$[6 \mid x]$	$y(x)$	OR
1	0	0	0	0	0	0	0
2	1	0	1	0	0	1	1

x	A	B	$[2 \mid x]$	$[3 \mid x]$	$[6 \mid x]$	$y(x)$	OR
3	0	1	0	1	0	1	1
6	1	1	1	1	1	1	1

Circuit interpretation. The OR gate operates by *inclusion-exclusion via the Möbius function*. The contributions from addresses 2 and 3 count each bit individually; the contribution from address 6 corrects for double-counting when both are present. The Möbius signs provide exactly the alternating $+, +, -$ pattern required by inclusion-exclusion.

5.4 NAND Gate

Goal: $\mathrm{NAND}(A, B) = \mathrm{NOT}(\mathrm{AND}(A, B)) = 1 - [6 \mid x]$.

Database:

n	O_n	C_n	$\mu(n)$	$\mu \cdot O$	Role
1	1	1	$+1$	$+1$	Reference $(+1)$
6	-1	1	$+1$	-1	Subtract $[6 \mid x]$

Verification:

x	A	B	$[6 \mid x]$	$y(x)$	NAND
1	0	0	0	1	1
2	1	0	0	1	1
3	0	1	0	1	1
6	1	1	1	0	0

5.5 Functional Completeness

Theorem 5.1. Any Boolean function can be implemented as a database program in the master equation.

Proof. The NAND gate is implementable (Section 5.4), and NAND is functionally complete: any Boolean function can be constructed from NAND gates alone (Sheffer [3]). □

5.6 Gate Summary

Gate	Database entries	Formula
$\mathrm{NOT}(A)$	$D(1) = (1, 1)$, $D(2) = (1, 1)$	$1 - [2 \mid x]$
$\mathrm{AND}(A, B)$	$D(6) = (1, 1)$	$[6 \mid x]$
$\mathrm{OR}(A, B)$	$D(2) = (-1, 1)$, $D(3) = (-1, 1)$, $D(6) = (-1, 1)$	$[2 \mid x] + [3 \mid x] - [6 \mid x]$
$\mathrm{NAND}(A, B)$	$D(1) = (1, 1)$, $D(6) = (-1, 1)$	$1 - [6 \mid x]$

6. The Möbius Function as Inclusion-Exclusion

The central structural insight of the architecture is that the Möbius function *naturally* provides the correct signs for Boolean computation. This is not a coincidence or a design choice — it is a direct consequence of the algebraic structure of the Euler product inverse.

6.1 The Connection

The Euler product inverse is:

$$\frac{1}{\zeta(s)} = \prod_p (1 - p^{-s}) = \sum_{n=1}^{\infty} \frac{\mu(n)}{n^s}.$$

Expanding the finite product $\prod_{p \in S}(1 - p^{-s})$ for a finite set of primes $S = \{p_1, \ldots, p_k\}$:

$$\prod_{p \in S}(1 - p^{-s}) = 1 - \sum_i p_i^{-s} + \sum_{i<j} (p_i p_j)^{-s} - \cdots$$

The coefficient of n^{-s} for squarefree n with prime factors in S is exactly $\mu(n) = (-1)^{\text{number of prime factors}}$. This is the inclusion-exclusion expansion: the alternating signs add individual contributions, subtract pairwise overlaps, add back triple overlaps, and so on.

6.2 How This Manifests in the Gates

In the OR gate (Section 5.3), the inclusion-exclusion structure appears directly:

$$\text{OR}(A, B) = [2 \mid x] + [3 \mid x] - [6 \mid x].$$

The three terms correspond to: - $+[2 \mid x]$: include A - $+[3 \mid x]$: include B - $-[6 \mid x]$: exclude overlap $A \cap B$

The signs arise because: $\mu(2) \cdot O_2 = (-1)(-1) = +1$; $\mu(3) \cdot O_3 = (-1)(-1) = +1$; $\mu(6) \cdot O_6 = (+1)(-1) = -1$. The Möbius function provides the alternating sign structure; the database output $O_n = -1$ serves as a uniform scaling that, combined with μ, yields the inclusion-exclusion coefficients.

In the NOT gate (Section 5.1), the $\mu(2) = -1$ sign produces the *phase interference* that subtracts the input from a reference. The NOT operation $1 - A$ is the simplest case of inclusion-exclusion: the complement of a single set.

6.3 The General Pattern

For a Boolean function on k input bits encoded at primes $p_1, \ldots, p_k$, the relevant addresses are the squarefree products of subsets of $\{p_1, \ldots, p_k\}$. There are 2^k such products (including $n = 1$). The Möbius function assigns $(-1)^j$ to products of j primes, which is exactly the inclusion-exclusion sign for subsets of size j.

This is the same structure that appears in the Euler product: the factors $(1 - p^{-s})$ each contribute a "subtract" term, and their product expands via inclusion-exclusion into the Möbius sum $\sum \mu(n)/n^s$.

---

7. Native Computation and the Euler Product

A remarkable feature of the architecture is that its most fundamental computation — the partial Euler product — is itself a program in the master equation.

7.1 The Partial Euler Product

For an input set S, define the *coverage product*:

$$R(S) = \prod_{p \in S} \left(1 - \frac{1}{p^2}\right).$$

This measures what fraction of "coprimality structure" the input set accounts for. As S grows to include all primes, $R(S) \to 6/\pi^2 = 1/\zeta(2)$, the coprimality probability [2].

7.2 The Möbius Sum Equivalence

Theorem 7.1. The coverage product equals the restricted Möbius sum:

$$R(S) = \prod_{p \in S} \left(1 - \frac{1}{p^2}\right) = \sum_{\substack{n \text{ squarefree} \\ \text{prime factors} \subseteq S}} \frac{\mu(n)}{n^2}.$$

Proof. This is the Euler product identity $\prod_p (1 - p^{-s}) = \sum_n \mu(n)/n^s$ restricted to primes in S. The restriction is valid because the product over a finite set of primes expands to a finite sum over squarefree integers whose prime factors lie in S. □

7.3 Native Computation as a Program

The Möbius sum for $R(S)$ can be written in the master equation form:

$$R(S) = \sum_n \mu(n) \cdot \frac{1}{n^2} \cdot [n \mid S] \cdot 1$$

This is exactly the master equation $y(S) = \sum_n \mu(n) \cdot O_n \cdot [n \mid S] \cdot P_n(S)$ with the "native database": - $O_n = 1/n^2$ for all squarefree n with prime factors in S; - $C_n = 1$ (trivial condition, so $P_n(S) = 1$ always).

Interpretation. The "operating system" — the Euler product structure that generates $R(S)$ — is itself a program running in its own architecture. The Möbius signs, the divisibility-based activation, and the summation structure are shared between the native computation and the Boolean gate programs of Section 5. This self-referential unity is a structural feature of the Euler product, not an engineered design.

7.4 Coverage Ratio

Define the *coverage ratio* $\rho(S) = \zeta(2) \cdot R(S) = (\pi^2/6) \cdot R(S)$.

Properties: - $R(\emptyset) = 1$, $\rho(\emptyset) = \pi^2/6 \approx 1.6449$. - $R(\text{all primes}) = 6/\pi^2 \approx 0.6079$, $\rho(\text{all primes}) = 1$. - $\rho(S) \to 1$ as $S \to \{\text{all primes}\}$.

The ratio $\rho(S)$ measures the fraction of the full Euler product that the input set S accounts for.

Worked example. For $S = \{2, 3, 7, 11\}$ (binary input `11011`):

$$R(S) = \frac{3}{4} \cdot \frac{8}{9} \cdot \frac{48}{49} \cdot \frac{120}{121} = 0.6477, \qquad \rho(S) = \frac{\pi^2}{6} \cdot 0.6477 = 1.0654.$$

---

8. Conditional Logic and Gate Composition

The processor function $P_n(S)$ enables conditional tests *independent* of the activation address. This allows the activation gate and processor gate to form a two-gate series circuit implementing conditional logic.

8.1 Example: "If A AND D, then output 100"

We want: output 100 when both bit A (prime 2) and bit D (prime 7) are ON.

Implementation. Use address $n = 2$ (activation tests $2 \mid x$) with condition $C = 7$ (processor tests $7 \mid x$) and output $O = -100$ (sign-corrected by $\mu(2) = -1$):

$$\text{Term} = \mu(2) \cdot (-100) \cdot [2 \mid x] \cdot P(7, x) = (-1)(-100)[2 \mid x][7 \mid x] = 100 \cdot [2 \mid x] \cdot [7 \mid x].$$

Database:

n	O_n	C_n	$\mu(n)$	Role
2	-100	7	-1	Output 100 when $2 \mid x$ AND $7 \mid x$

Verification:

x	A	D	$[2 \mid x]$	$P(7, x)$	$y(x)$	Expected
1	0	0	0	0	0	0
2	1	0	1	0	0	0
7	0	1	0	1	0	0
14	1	1	1	1	100	100

Circuit interpretation. The activation gate $[2 \mid x]$ and processor gate $P(7, x)$ form a series circuit: signal flows only when both gates pass. This is the computational analogue of two resistors in series — both must have zero resistance for current to flow.

8.2 Composing Multiple Conditions

More complex conditional logic is achieved by combining multiple database entries. For example, a program that outputs 100 when A AND D, and separately outputs 200 when B AND D, uses:

n	O_n	C_n	$\mu(n)$	Effect
2	-100	7	-1	$+100$ when $A \wedge D$
3	-200	7	-1	$+200$ when $B \wedge D$

The master equation sums all activated terms, with μ providing the correct signs. When multiple programs activate simultaneously, the Möbius-weighted sum automatically handles interactions via inclusion-exclusion.

---

9. Application: The Factor-Free Fingerprint

As an application of the framework, we derive an identity that connects the Euler product structure to a computation requiring only GCD operations.

9.1 The Factorisation Fingerprint

Definition 9.1. For a positive integer N with prime factors $S_N = \{p : p \mid N\}$, the *factorisation fingerprint* is:

$$\rho(N) = \zeta(2) \cdot \prod_{p|N} \left(1 - \frac{1}{p^2}\right) = \frac{\pi^2}{6} \cdot R_{S_N}(2).$$

The fingerprint depends only on which primes divide N, not on their multiplicities: $\rho(6) = \rho(12) = \rho(72)$ since all have prime factor set $\{2, 3\}$.

Properties. For all positive integers N: $1 < \rho(N) \leq \zeta(2)$, with $\rho(N) \to \zeta(2)$ as $N \to$ large prime, and $\rho(N) \to 1$ as $N \to$ primorial.

9.2 The Factor-Free Identity

Definition 9.2 (Localised Zeta Function). For integer N and $\mathrm{Re}(s) > 1$:

$$\zeta_N(s) = \sum_{\gcd(n,N)=1} n^{-s}.$$

This sums over all positive integers coprime to N.

Theorem 9.3 (Factor-Free Fingerprint). The fingerprint equals the localised zeta at $s = 2$:

$$\rho(N) = \zeta_N(2) = \sum_{\gcd(n,N)=1} \frac{1}{n^2}.$$

Proof. The localised zeta satisfies $\zeta_N(s) = \zeta(s) \cdot \prod_{p|N}(1 - p^{-s})$. This follows from the Euler product: summing over integers coprime to N excludes all multiples of primes dividing N, which removes the corresponding factors from the Euler product. At $s = 2$: $\zeta_N(2) = \zeta(2) \cdot R_N(2) = \rho(N)$ by Definition 9.1. □

9.3 Computational Significance

The crucial observation is that $\gcd(n, N)$ is computable via the Euclidean algorithm in $O(\log N)$ time *without knowing the factors of* N. Therefore, $\rho(N)$ can be approximated by summing $1/n^2$ over integers n coprime to N, using only GCD computations — no factorisation required.

Numerical verification (truncation at $n_{\max} = 10{,}000$):

N	Factors	ρ (exact)	$\zeta_N(2)$ (truncated)	Error
30	$\{2, 3, 5\}$	1.0528	1.0527	$< 0.01\%$
35	$\{5, 7\}$	1.5469	1.5468	$< 0.01\%$
143	$\{11, 13\}$	1.6217	1.6216	$< 0.01\%$
2310	$\{2, 3, 5, 7, 11\}$	1.0228	1.0227	$< 0.01\%$

Remark. While the identity $\rho(N) = \zeta_N(2)$ provides a factor-free *computational path* to the fingerprint, it does not provide a practical factorisation algorithm. Computing $\zeta_N(2)$ to sufficient precision to distinguish fingerprints requires summing $O(n_{\max})$ terms, where $n_{\max}$ must grow with the size of N's smallest prime factor. The identity is of theoretical rather than practical interest.

---

10. Discussion and Open Problems

10.1 Summary

We have presented a computational architecture in which Boolean logic emerges from the multiplicative structure of the integers. The Möbius function provides the routing signs; divisibility provides activation; the master equation provides a uniform framework for all gates; and functional completeness follows from the implementability of NAND.

The deepest structural feature is the identity between the Möbius inclusion-exclusion signs and the coefficients required for Boolean logic. The OR gate's database entries produce $+1, +1, -1$ coefficients — exactly the inclusion-exclusion pattern — not because we engineered this, but because the Euler product inverse $\prod(1 - p^{-s}) = \sum \mu(n)/n^s$ *is* the inclusion-exclusion expansion over prime sets.

10.2 Limitations

The architecture as presented implements *combinational* (feed-forward) Boolean logic. It does not include a mechanism for feeding output back as input, which would be required for sequential computation (memory, loops, state machines). The functional completeness of Section 5.5 is completeness for Boolean *functions*, not for Turing computation.

Comparison with Paper 1. The Turing completeness established in [1] uses a different mechanism (Gödel-encoded counter machines) that operates *within* the number-theoretic framework but *outside* the master equation architecture. Whether the master equation itself can be extended to support looping and sequential logic is an open question.

10.3 Open Problems

1. **Feedback and looping.** Can the architecture be extended so that the output $y(S)$ is fed back as a new input? If so, does the resulting system achieve Turing completeness? A natural approach would be to define a sequence $S_0, S_1, S_2, \ldots$ where S_{t+1} is determined by $y(S_t)$ via some encoding. The convergence and halting properties of such a system are unexplored.

2. **Computational complexity.** What is the cost of evaluating $y(S)$ for a database of size $|D|$? A naive evaluation requires $O(|D|)$ GCD computations. For databases implementing k-input Boolean functions, $|D| \leq 2^k$. Does the Möbius structure permit more efficient evaluation?

3. **The continuous processor.** The generalised processor $P(C, x) = \ln(\gcd(C, x))/\ln(C)$ produces fractional values for partial matches (Section 3.3). Can this be exploited for non-Boolean computation — for instance, fuzzy logic, probabilistic inference, or continuous optimisation?

4. **Advantage or reformulation?** Does the architecture offer any computational or conceptual advantage over conventional Boolean circuit models, or is its value primarily as a reformulation that makes the number-theoretic structure of Boolean logic visible? The fact that the Möbius function "naturally" provides inclusion-exclusion signs is aesthetically appealing, but it remains to be seen whether this perspective leads to new results.

5. **Multi-output programs.** Can the architecture be extended to compute vector-valued outputs, allowing a single database evaluation to produce multiple bits simultaneously?

References

[1] Pisani, A. (2026). A unified algebraic framework for number theory, Boolean logic, and circuit topology via the logarithmic isomorphism. *(Companion paper.)*

[2] Euler, L. (1734). De summis serierum reciprocarum. *Commentarii academiae scientiarum Petropolitanae*, 7, 123–134.

[3] Sheffer, H. M. (1913). A set of five independent postulates for Boolean algebras. *Transactions of the American Mathematical Society*, 14(4), 481–488.

[4] Minsky, M. L. (1967). *Computation: Finite and Infinite Machines.* Prentice-Hall.

[5] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423.

[6] Gödel, K. (1931). Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik*, 38, 173–198.

[7] Hardy, G. H. & Wright, E. M. (1979). *An Introduction to the Theory of Numbers* (5th ed.). Oxford University Press.

[8] Aczél, J. (1966). *Lectures on Functional Equations and Their Applications.* Academic Press.

[9] Apostol, T. M. (1976). *Introduction to Analytic Number Theory.* Springer.

[10] Davey, B. A. & Priestley, H. A. (2002). *Introduction to Lattices and Order* (2nd ed.). Cambridge University Press.

Appendix A: Complete Gate Verification Tables

For reference, we provide the full verification of all gates across all four input combinations for two-bit inputs (A at prime 2, B at prime 3).

A.1 NOT(A)

x	Bits	$[1 \mid x]$	$[2 \mid x]$	$\mu(1) \cdot 1 \cdot [1 \mid x]$	$\mu(2) \cdot 1 \cdot [2 \mid x]$	y	Expected
1	A=0, B=0	0	+1	0	1	1	
2	A=1, B=0	1	+1	−1	0	0	
3	A=0, B=1	0	+1	0	1	1	
6	A=1, B=1	1	+1	−1	0	0	

A.2 AND(A, B)

x	Bits	$[6 \mid x]$	$\mu(6) \cdot 1 \cdot [6 \mid x]$	y	Expected
1	00	0	0	0	0
2	10	0	0	0	0
3	01	0	0	0	0
6	11	1	+1	1	1

A.3 OR(A, B)

x	Bits	$[2 \mid x]$	$[3 \mid x]$	$[6 \mid x]$	Terms	y	Expected
1	00	0	0	0	0+0−0	0	0
2	10	1	0	0	1+0−0	1	1
3	01	0	1	0	0+1−0	1	1
6	11	1	1	1	1+1−1	1	1

A.4 NAND(A, B)

x	Bits	$[6 \mid x]$	$\mu(1) \cdot 1 - \mu(6) \cdot 1 \cdot [6 \mid x]$	y	Expected
1	00	0	$1 - 0$	1	1
2	10	0	$1 - 0$	1	1
3	01	0	$1 - 0$	1	1
6	11	1	$1 - 1$	0	0

Appendix B: Notation

Symbol	Meaning
S	Input prime set
$y(S)$	Master equation output
$\mu(n)$	Möbius function
O_n	Database output value at address n
C_n	Condition associated with program at address n
$[n \mid S]$	Activation indicator: 1 if all prime factors of n are in S
$P_n(S)$	Processor function: tests condition C_n against S
D	Database: finite set of (n, O_n, C_n) triples
$R(S)$	Coverage product: $\prod_{p \in S}(1 - 1/p^2)$
$\rho(S)$	Coverage ratio: $\zeta(2) \cdot R(S)$
$\zeta(s)$	Riemann zeta function
$\zeta_N(s)$	Localised zeta function: $\sum_{\gcd(n,N)=1} n^{-s}$
Ω_n	Informational resistance: $\ln(n)$
β	Binary-to-prime encoding function